

Project Journey Portfolio

StudyOS Thesis-Finder — working draft

Group submission • Dominique, Max, Valentin • University of Tübingen

Info-gathering version. Each section mirrors the assignment structure (What / Evidence / So what / Now what) and states what we already have from the repository. Everything marked in an **Open** box is a question the team still needs to answer — these are the gaps between what is documented and what the assignment explicitly asks for.

The journey logic in one paragraph (draft): We started with a matching platform (app + database) for thesis search. Early conversations with professors broke the core premise: the supply side won't feed a central platform, and a curated database rots. We reframed the problem from "matching" to the student's cold start, pivoted to a database-free agent skill that searches the live web, built and self-evaluated it against a plain-Claude baseline, and shipped it as a public, installable package. What's still missing is testing in the hands of real students — the protocol is ready, the test hasn't run yet.

1 Initial Project Idea

We solve X for Y by doing Z

We solve **topic discovery** for **thesis students** by **bringing open theses into one place**

Our initial idea was a full-stack web app for students and professors. The professors could easily list their recent thesis topics, while the students could search through the list of available topics and find their most suitable match. The motivation came from our own experience of searching and finding a suitable thesis topic. At the university, there are multiple different data sources like the official university website, Alma, and additionally most of the professors also have a private lab-based website. Our first hypothesis was that the major problem is the lack of a single source of truth. Therefore, we have planned and developed a first full-stack prototype, with a backend, a database, and a web frontend. Since we are in the age of AI, we also planned to integrate a fully custom AI agent with custom tool calls that could supervise students during the search process and the exploration of their own possibilities and goals for their thesis. There we fell into the trap that we, as software engineers, love to build complex systems, just because we can, not because it is actually needed.

Our initial target group was students and professors of the Department of Computer Science at the University of Tuebingen. We treated the students as the demand side and the professors as the supply side of our platform. After discussing the initial idea in the course, a lot of students thought that the idea was promising, because a lot of them faced the same problem in their bachelor studies. Common stories were that students were frustrated in the end because they didn't know about specific chairs, which might have been the perfect fit for their interests and skills. Another main point was that a lot of students reached out to professors, but didn't get a response. At this point, we hadn't done the user research for the supply side yet, so we only had the students' perspective and the perspective of Professor Gehler.

To mitigate this discrepancy, we planned a user study for professors, where they could give us their point of view, without rating the actual idea itself. We wanted to understand the reasons behind the missing thesis list and whether it was intended or just fell off due to the lack of time.

While the whole application got stripped apart during the process of the course, the initial idea was a good starting point and a problem worth solving.

2 Discovering and Understanding Users

Assignment asks for: user groups discovered and selected, how they were reached, the research approach, main insights, a persona or user story — plus what we learned that the initial idea alone couldn't tell us, and how that shaped the direction.

What we have

- Two user groups: students are the demand side, professors/chairs are the supply side a platform would depend on. We are in the middle and wanted to satisfy both sides.
- We interviewed a substantial number of professors via E-Mail. We sent 4 probe e-mails to professor we knew, then we automatically sent mails to all the professors in the department of computer science. This took place from late April till May. We received a lot of feedback, which we documented and analyzed. The main insights are summarized below. The E-Mails can be found in the Appendix A. The professors were contacted via the official university e-mail addresses. We sent out two different variants of the e-mail, depending on whether the chair already had a public list of topics or not. The professors were asked to give us their honest feedback on the current situation and the proposed idea. Main documented insights:
 - Roughly half rejected a central platform outright; only about a third were conditionally open to it. Reason for rejection was that the professors think that fitting a student to a thesis topic is a personal process and that they want to get to know the student first. One important quote "the old system was the exact opposite: Listing theses was cumbersome. The list wasn't maintained. And being among the few groups that listed on the central page usually just meant that the less engaged, less informed students would approach you first."
 - One professor reacted negatively to our e-mail and thought they were too demanding. One important lesson is that when you send out a request to professors, you should be very careful with the wording and the tone of the message.
 - A comparable topic-listing platform at the university had already failed — for incentive reasons, not UI reasons: nobody wants to list and maintain open topics. This caused us to shift our product without surveying students, since the supply side is the bottleneck.

So we learned that the best way to please professors is to not ask them to do much at all. They can still go on and update their own website, but we don't need their input necessarily.

- We also anticipated professors complaining about AI spam, so we instructed the skill to output a clear disclaimer in order to encourage students to at least modify the generated e-mails and drafts.

So what (draft)

- The initial idea could not have told us that the *supply side* is the bottleneck. Interviews revealed the platform premise fails regardless of how good the product is — participation and freshness are incentive problems, not engineering problems.

Now what)

- The main insight was that we could not work with a central list, but should focus on what is publicly available and on the students's side.
- This brought us to the idea of building an advisor that helps students, as detailed in the next section.

3 Defining the Problem

What we have

We at first tried to find a problem that fit our rough goal a problem we had and we knew from our own student life. Finding the right bachelor or master thesis can be difficult. At first we thought that the problem was that there was no central list of topics. With the feedback of professors we

knew that this was not a solution to the problem of finding a suitable thesis topic for students and it would not have been maintainable either.

So what (draft)

- We therefore discussed multiple pivots, however we still wanted to build an app, that would have been deployed. It would have been a full-stack app with a database and a backend.
- We discovered our new formulation of the problem : We solve the problem of finding a suitable thesis topic for students by providing a tool that helps students find and reflect on their thesis by providing information and idea via an LLM.
- For example the app would be able to ingest the transcript of records and then try to build a profile that would be used to find suitable thesis topics.

Now what (draft)

- We now had a clear idea of the problem and started building an app. We wanted to assess the students based on different axis, like interest or skill. We also scraped papers by the professors and their chairs. This was all done so we might use local models or would have a big database that we could use to answer the students' questions.
- You can find the app in the git logs of our repository. However since we later decided to not use an app but a skill and then even later pivoted to a skill without any database (even in text form) whatsoever, we have deleted it from the current iteration of the repository.
- What stuck from this phase was the main idea : Advising students. Also we carried the idea of building a profile of the student first

4 Proposing a Solution

NEEDS TO BE DELETED AFTER THE PORTFOLIO IS DONE

Assignment asks for: solution concept, value proposition, alternatives considered, feature prioritization, and the fit between users, problem, and solution.

As described in Section 3, we have changed the problem definition from a tool for professors and students to a tool for students only. After we've evaluated the answers from the professors, it became quite clear that they are not interested in the centralized app solution. Mainly because they don't want the overhead, and they want to get to know the student to find a suitable thesis. Since the initial idea was built on the supply of the professors, we had to rethink our approach, because the problem for the students still exists. The core problem is not the absence of a central thesis-topic list. Students often lack the context to decide which chair or company fits their interests, methods, skills, constraints, and thesis style. This makes thesis search highly individual: students need guided matching, not just another static list. Since this problem is highly individual and needs deep information about the user, we thought about a private assistant.

We initially stuck to the idea of a full-stack app, with a parser that extracts the information from the professors' websites and then presents it to the user. After we presented our idea, the resulting reaction was again: "Why not just a Skill?" For us, it was quite new that we "just" use a markdown file to describe the task and an LLM will do the rest. After further discussion in the team, we really embraced the idea of a Skill, because a Skill can be inspected and adapted through plain Markdown, lowering the barrier for future maintainers.

Additionally, we don't have to think about security, privacy, deployment, and cloud billing since this is already handled by your LLM provider of choice. This realization then led to the initial implementation of our first skill architecture, with a markdown database of all possible professors, including their recent papers, teams, and their research topics. The skill then used this information to answer student-specific questions in a conversational way. During this phase, we thought deeply about what information is actually necessary to provide high-quality answers. We

have developed a 6-dimensional scheme, which needs to be answered before the student gets any recommendation from the LLM. The 6 dimensions are

Dimension	Name	Description
1	Interests	Core research areas or topics
2	Methods	How the student wants to work, e.g. empirical, qualitative, computational
3	Domain	Application field, e.g. healthcare, education, finance, automotive
4	Thesis style	Preferred output type, e.g. experimental, theoretical, systems, analysis, survey, mixed
5	Skills	Concrete tools or competencies, e.g. Python, ML frameworks, lab methods
6	No-gos	Hard exclusions, e.g. hardware setup, pure proofs, large-scale software engineering

In our opinion, those 6 dimensions are the most important cornerstones for a successful thesis search.

The biggest task here was "Why should a student use our skill and not just plain Claude/Codex?" The answer to this question is that our skill is not a "give me the answer as fast as possible", but it emulates a session with a professional study coordinator that explicitly tries to find the best possible match for the individual student.

After we have implemented this orchestrated multi-dimensional skill, we've tested it thoroughly and recognized that the markdown database needs to be maintained in some way, which we wanted to avoid in the first place. We extended our current prototype and built a database-less approach, where we are utilizing the commonly used web search tool of the LLM providers to find the most suitable professors and their research topics. This approach has the advantage that we don't have to maintain a database, which can be costly and error-prone if, e.g., new Professors join or old ones leave the Department of Computer Science. The biggest disadvantage here was that the search tool introduces even more uncertainty and non-deterministic behavior into the skill. To mitigate this, we have built explicit search templates to get the most relevant information from the web search. We conducted a field study and let the course test both skills, the one with the markdown database and the one without. After evaluating the results, we received feedback that our skill without the database is more useful. This led to the final decision of removing the Markdown database and using the slimmer version with the minimal maintenance overload. Another finding during the field study was that some students also want to do a thesis in a company. Therefore, we extended the skill to also provide suitable thesis topics from companies in Baden-Württemberg. Since the skill is more uncertain, and a session takes longer than 30 minutes, we decided to use some automated testing. Therefore, we built evals as well as an orchestrated skill set to run end-to-end simulations with an LLM-as-a-judge to evaluate the actual performance. These automated tests gave a major boost in confidence and implementation speed, since the agent can now fully simulate actual conversations. In order to make this representative, we designed multiple fictitious students from different backgrounds and with different skill sets and answer styles.

Our final product is a low-maintenance, database-free skill package that guides students from vague thesis interests to evidence-backed university or company options, without requiring professors to maintain yet another platform.

5 Building

The product itself is submitted separately in the Product Artifact ZIP — per the assignment, this section only refers to it.

One connecting sentence (draft): the build followed the priority order from Section 4 — university discovery first, then a company arm on the same no-database principle, then consolidation

into a single entry-point skill; the ZIP contains the skill package, install guide, and evaluation harness.

6 Testing

Assignment asks for: what was tested, who tested it, how, main feedback, and problems / surprises / confirmations — with test notes and quotes as evidence.

What we have

- Extensive **self-run evaluation** (this is real testing and should be presented as such, but it is not user testing):
 - Recall/precision measured per faculty against hand-curated ground truth, with the skill scoring far above a plain-Claude baseline.
 - A steering test showing that different student profiles genuinely produce different option maps (not one generic answer).
 - An independent, deliberately skeptical review that challenged our own “it works” verdict and produced a punch list — including catching that some of our numbers weren’t as clean as claimed. Honest material, good for the portfolio.
 - A surprise worth telling: our first eval design was circular (the fixture setup let the system be graded on data it had effectively seen) — we caught it and moved to live evals. That’s a genuine “problem discovered through testing.”
- **Informal in-between tests** with people around the project happened, including a hands-on session with the course professor that produced direct feedback — but these were never written down properly.
- **Real student beta test: not yet run.** Protocol, capture sheet, and recruiting message are written and ready; nobody outside the project has used the tool independently yet. The portfolio must state this honestly.

Evidence to attach

- Evaluation summary and numbers: docs/thesis-report/03-hardening-and-evaluation/README.md, findings/no_db_universal_skill/2026-07-03-eval-aggregate-scorecard.md
- The one clean measurement on the current architecture: findings/no_db_universal_skill/2026-08-08-theology-blind-run.md
- Skeptical review: findings/no_db_universal_skill/2026-07-05-fable-1.0-readiness-review.md
- Beta-test protocol (ready, unexecuted): findings/no_db_universal_skill/2026-08-08-beta-test-protocol.md

So what / Now what

- Depends on the open items below — this section can’t be finished until the informal tests are reconstructed and (ideally) the beta test has run.

Open — needs the team

- Reconstruct each informal test as a proper record: who tested (role is enough, no names needed), what they were asked to do, what they actually said — ideally one verbatim quote each — and what surprised us or confirmed our assumptions.
- The professor’s hands-on session produced the single most important piece of feedback in the whole project (value = reflection, not proposals). Reconstruct that session concretely: what was demoed, what was said, what changed because of it.
- Decide: do we run the beta test before submission? If yes — results go here. If no — the portfolio

says the protocol is ready, explains why it hasn't run, and Section 7 carries it as the next step. Either is defensible; silently omitting it is not.

7 Iteration and Next Steps

Assignment asks for: changes made from feedback, changes planned but not done, remaining limitations, distribution/shipping strategy, maintenance/handover strategy, and the possible role of AI in maintaining or evolving the project.

What we have

- Changes made based on feedback (pick the 2–3 clearest for the portfolio):
 - The pivot itself — the largest feedback-driven change in the project.
 - The professor's reframing ("the value is the student's reflection") was built into the product: the skill now records the student's thesis self-understanding before and after the search.
 - Multiple concrete skill fixes that came directly out of eval findings (e.g. verification rules that were losing correct candidates).
- Changes planned but not implemented: the real student beta test; re-measuring the faculties whose numbers predate the architecture change; a designed-but-never-run experiment testing our central claim (that the skill's advantage comes from curated local scope and would erode as scope grows).
- Remaining limitations: only one clean measurement on the current architecture; company arm weaker than the university arm; coverage honesty is a feature but also a limit.
- Distribution/shipping (documented, real): the skill package is publicly released on GitHub with an install guide, and a cold install was tested. Distribution = "install these markdown folders," which is the whole point.
- Maintenance/handover: a survival plan exists — because the product is markdown-only (no database, no backend, no API keys), almost nothing can rot; main risks are the skill format changing, university sites restructuring, and rules going stale. Estimated cost: a couple of hours per semester. Ownership options are ranked, **but no owner has been named**.
- Role of AI in maintenance — we have an unusually strong answer: the repo ships an operating guide written *for future AI agents* (AGENTS.md), so the intended maintainer of this AI-built product is itself an AI agent, with a human owner spending ~2h/semester steering it. This maps exactly onto the assignment's question and should be featured, not buried.
- We also extended this formula of an interview and then building a profile to the topic of finding jobs(not only for students) <https://github.com/ValentinJSchmidt/german-career-compass-skills>. While almost untested and not as polished, it is a good proof of concept how the abstract idea of the skill can be applied to other domains as well. Using coding agents this is made particularly easy since the general structure can be adapted quite well in a few-shot manner. Being able to craft some useful skills for this almost on the fly can prove very useful if you chose to use LLM for decisions like this.

Evidence to attach

- Open work and rejected ideas: docs/thesis-report/04-open-work/README.md
- Survival/maintenance plan: findings/no_db_universal_skill/2026-08-09-survival-and-maintenance-plan.md
- Public release + install guide: GitHub release page, INSTALL.md
- Agent maintainer guide: AGENTS.md

So what (draft)

- Candidate for the most important iteration learning: every major improvement came from an outside signal (interviews, professor feedback, the skeptical review) — never from us admiring our own output. To be confirmed/sharpened by the group.
- We have tried to reach out to students as well and even set up a survey on <https://www.soscisurvey.de/> to get their feedback on the initial idea and sent it through the mailing list of the fachschaft. However response was slow. Therefore we were limited to informal conversations with fellow students and friends. All of them gave us positive feedback. One student was also not from Tübingen and from the economic faculty, so we also had some expanded feedback. Most of them are not on the level that they trust AI completely, however and take the recommendations with a grain of salt. However this is not at all a problem, since we also do not want to give students a final answer, but rather a tool that helps them reflect their own interests and skills.

Now what (draft)

- If the project continued, the next meaningful step is putting the tool in real students' hands (the beta test) — because it's the only claim in the whole journey we haven't tested yet.

Open — needs the team

- Ownership must be resolved for the portfolio: does one of us take the ~2 h/semester maintenance role, does it go to someone at the university, or do we honestly write “unowned — and here is what that means for the product's lifespan”? Any of the three works; the gap can't stay silent.
- Has anything changed since the last documented project state (new runs, new feedback, new decisions)? The portfolio should reflect where we actually are at submission, not where the repo log stops.
- Confirm or replace the “most important iteration learning” draft above — this should be a sentence the whole group stands behind.

Submission checklist — current status

Checklist item	Status
First problem idea	✓ covered (Section 1)
Initial project idea / proposal	✓ covered — <i>needs one early artifact</i>
Relevant users / user groups	✓ covered — <i>needs persona + student-side account</i>
Initial and refined problem definitions	● content there — <i>needs the two one-sentence versions</i>
Proposed solution + value proposition	● content there — <i>needs the agreed one-sentence value prop</i>
High-level product description	✓ covered (Section 5 + ZIP)
User testing and feedback	✗ weakest point — <i>informal tests must be written up; beta decision needed</i>
Iteration / planned changes	✓ covered (Section 7)
Shipping / access / maintenance strategy	✓ covered — <i>except the unresolved owner</i>
Concrete evidence, not just claims	✓ strong — repo is full of citable artifacts
Clear and concise structure	pending — this draft becomes 4–6 pages once gaps close

A Interview Guide — Professors

The professor research referenced in Section 2 was conducted by e-mail. We sent two variants depending on whether the chair already published thesis topics or not; both are reproduced verbatim below.

Mode: e-mail • **Period:** *[dates]* • **Contacted:** *[n]* chairs • **Responses:** *[n]*

Mail A — chair already lists topics publicly

Subject: Thesis topic listings at your chair – your experience & a new approach

Dear [Name],

We are Master’s students in Machine Learning at the University of Tübingen, working on a project supervised by Prof. Peter Gehler to improve how thesis topics are shared and managed across chairs in our department.

We noticed that your chair already lists thesis topics publicly — which is genuinely rare and makes your perspective especially valuable to us.

We have a few concrete questions about your current experience:

1. How much effort does it take to keep your topic list up to date — is it manageable, or does it create noticeable overhead?
2. Do topics sometimes stay listed past their actual availability, or get delayed because updating feels like extra work?
3. How do you currently decide when and how to update the list — is there a process, or does it happen ad hoc?

We’re developing a concept we’d love your honest reaction to: a centralized platform where chairs manage their topics and receive periodic reminders to review and update their listings — keeping the overhead as low as possible.

A few follow-up questions on that:

4. Is that something your chair would actually want to use — and if not, what would need to be different for it to make sense?
5. Are there concerns around data privacy or how a shared system would handle your content that we should be aware of?

A brief written reply would already help us enormously. If anything is easier to discuss directly, we’re happy to find a time.

Best regards, [Your name] (Project supervised by Prof. Peter Gehler)

Mail B — chair has no public topic listings

Subject: Thesis topics at your chair – a few questions about going public

Dear [Name],

we are Master’s students in Machine Learning at the University of Tübingen, working on a project supervised by Prof. Peter Gehler on how thesis topics are currently — and often not — shared across chairs in our department.

We noticed that your chair doesn’t currently list thesis topics publicly, and we’re genuinely trying to understand why before drawing any conclusions. All answers are useful to us, including “we don’t want to.”

A few concrete questions:

1. Is the absence of public listings a deliberate choice — for example, because you prefer students to reach out directly — or more a matter of no good tool existing to make it easy?

2. Are there topics internally that would in principle be suitable for students, or do topics at your chair tend to emerge from ongoing research rather than being defined in advance?
3. If maintaining a public list has felt like too much overhead in the past: what specifically makes it costly — writing the descriptions, keeping them current, or something else?

We're developing a concept we'd love your honest reaction to: a centralized platform where chairs manage their topics and receive periodic reminders to review and update their listings — keeping the overhead as low as possible.

4. Is that something your chair would actually want to use — and if not, what would need to be different for it to make sense?
5. Are there concerns around data privacy or how a shared system would handle your content that we should be aware of?

We're not trying to push a solution — we want to know whether a problem worth solving actually exists here from your side. A brief written reply is everything we need. Happy to chat directly if that's easier.

Best regards, [Your name] (Project supervised by Prof. Peter Gehler)